

<#

Redact-FileContent.ps1

Unified replacement/redaction engine for files, recursively through subfolders.
All modes below share the same file-processing, dry-run and backup logic.

1. Literal/regex replacements (\$Replacements below) - always applied.
Each key is a regex pattern, each value is what to replace it with.
(Companion to Search-FilesForWord.ps1 - use that one first to preview matches.)
2. -AnonymizeSwitchNames - finds "S"/"s" followed by letters/digits/special characters and ending in "nn" (e.g. S-DC1-01nn), anywhere in a line - including glued to a prefix like "Uplink_to_S-DC1-01nn" - and replaces just that part with a sequential generic placeholder (SWITCH001, SWITCH002, ...).
3. -AnonymizeUsernames - finds "user" followed directly by digits (e.g. "user22351"), anywhere in a line - including glued to other text like "PC_user22351" or "user10023_workstation" - and replaces just that part with a sequential placeholder (USER001, USER002, ...).
4. -AnonymizeIPs - finds IPv4 addresses and replaces each with a sequential placeholder (IP001, IP002, ...).
5. -StripPortDescriptions - removes the description text from interface lines such as "GigabitEthernet1/0/1 - TO_SWITCH001", leaving just the port name ("GigabitEthernet1/0/1"). Covers GigabitEthernet, TenGigabitEthernet, TwentyFiveGigabitEthernet, FastEthernet and Ethernet ports.

For modes 2-4, the same original value always maps to the same placeholder within a single run (each category has its own counter), but nothing is persisted between runs - running the script again assigns fresh numbers.

All write modes support -DryRun (report what would change, touch nothing) and -Backup (copy each modified file's original content aside before overwriting it).

REPORTING:

- * Every changed line is printed under the file it belongs to, with its line number, how many matches it contained, and the line before ("-") and after ("+") the replacement.
- * Each file reports its match count, and the run ends with the total number of matches and changed lines across all files.
- * Folders and files that could not be opened (typically "access denied") are not skipped silently - they are listed at the end with their path and reason.
- * -ExportMappings writes the anonymization mappings to an Excel workbook with three tabs - Switches, Users and IP Addresses - each listing the original value against the generic name it was replaced with. The workbook is built as Open XML directly, so neither Excel nor any module has to be installed.

USAGE EXAMPLES:

```
# Preview the literal replacements only, no files modified
.\Redact-FileContent.ps1 -Path "D:\" -DryRun
```

```
# Apply literal replacements for real, keeping backups
.\Redact-FileContent.ps1 -Path "D:\" -Backup
```

```
# Anonymize switch names, usernames and IPs, and strip port descriptions
.\Redact-FileContent.ps1 -Path "D:\" -AnonymizeSwitchNames -AnonymizeUsernames -AnonymizeIPs -StripPortDescriptions -Backup
```

```
# Anonymization only - skip the literal $Replacements table
.\Redact-FileContent.ps1 -Path "D:\" -AnonymizeIPs -Replacements @{} -DryRun
```

```
# Limit to specific file types
.\Redact-FileContent.ps1 -Path "D:\Logs" -Include *.log,*.txt -AnonymizeUsernames -DryRun
```

```

# Anonymize everything and save the mapping tables as an Excel workbook
.\Redact-FileContent.ps1 -Path "D:\\" -AnonymizeSwitchNames -AnonymizeUsernames -AnonymizeIPs -ExportMappings

# Same, choosing where the workbook is written
.\Redact-FileContent.ps1 -Path "D:\\" -AnonymizeIPs -ExcelPath "C:\Temp\mappings.xlsx" -DryRun
#>

[CmdletBinding(SupportsShouldProcess = $true)]
param(
    # Root folder to process. Defaults to the current directory.
    [string]$Path = ".",

    # Optional file name filters (wildcards), e.g. *.txt, *.log, *.csv.
    # Leave empty to process every file.
    [string[]]$Include = @("*"),

    # Ordered map of regex pattern -> replacement text, applied on every line.
    [System.Collections.Specialized.OrderedDictionary]$Replacements = [ordered]@{
        "ass" = "aaa"
    },

    # Anonymize switch/device name tokens matching -SwitchNamePattern.
    [switch]$AnonymizeSwitchNames,
    [string]$SwitchNamePattern = '[Ss][^\s]*nn(?:\S)',
    [string]$SwitchNamePrefix = "SWITCH",

    # Anonymize username tokens matching -UsernamePattern.
    [switch]$AnonymizeUsernames,
    [string]$UsernamePattern = 'user\d+',
    [string]$UsernamePrefix = "USER",

    # Anonymize IPv4 addresses.
    [switch]$AnonymizeIPs,
    [string]$IPPattern = '(?<\d)(?:(?:25[0-5]|2[0-4]\d|1?\d?\d)\.){3}(?:25[0-5]|2[0-4]\d|1?\d?\d)(?!\d)',
    [string]$IPPrefix = "IP",

    # Strip the description text off interface lines, e.g.
    # "GigabitEthernet1/0/1 - TO_SWITCH001" -> "GigabitEthernet1/0/1".
    [switch]$StripPortDescriptions,
    [string]$PortDescriptionPattern = '^\\s*(?<port>(?:TwentyFiveGigabitEthernet|TenGigabitEthernet|GigabitEthernet|FastEthernet|Ethernet)\d+(?:/\d+)*)\s*-\s*.*$',

    # Report matches and what would change, without modifying any file.
    [switch]$DryRun,

    # Copy each file's original content into -BackupFolder before overwriting it.
    [switch]$Backup,

    # Folder to hold backups when -Backup is used. Defaults to a timestamped
    # folder created next to -Path. Relative structure under -Path is preserved.
    [string]$BackupFolder,

    # Write the anonymization mappings to an Excel workbook with three tabs:
    # Switches, Users and IP Addresses (old value vs. the new generic name).
    [switch]$ExportMappings,

    # Where to write that workbook. Supplying it implies -ExportMappings.
    # Defaults to a timestamped .xlsx created next to -Path.
    [string]$ExcelPath
)

if (-not (Test-Path -LiteralPath $Path)) {
    Write-Error "Path not found: $Path"
    return
}

if ($Backup -and -not $BackupFolder) {

```

```

    $BackupFolder = Join-Path (Resolve-Path -LiteralPath $Path) "_backup_$(Get-Date -Format 'yyyyMMdd_HHmss')"
}

$resolvedRoot = (Resolve-Path -LiteralPath $Path).ProviderPath

# Enumeration errors are collected instead of being swallowed, so folders that
# could not be opened (typically "access denied") are reported at the end
# rather than being skipped silently.
$allFiles = Get-ChildItem -Path $Path -Include $Include -Recurse -File -ErrorAction SilentlyContinue -ErrorVariable accessErrors

$ignoreCase = [System.Text.RegularExpressions.RegexOptions]::IgnoreCase
$switchNameRegex = [regex]::new($SwitchNamePattern, $ignoreCase)
$usernameRegex = [regex]::new($UsernamePattern, $ignoreCase)
$ipRegex = [regex]::new($IPPattern)
$portDescriptionRegex = [regex]::new($PortDescriptionPattern, $ignoreCase)

# Original value (as first encountered) -> generic placeholder, one map per category.
# Consistent for this run only - not persisted between runs.
$switchNameMap = [ordered]@{}
$usernameMap = [ordered]@{}
$ipMap = [ordered]@{}
$switchNameIndex = 1
$usernameIndex = 1
$ipIndex = 1

function Get-PlaceholderName {
    param(
        [string]$OriginalValue,
        [System.Collections.Specialized.OrderedDictionary]$Map,
        [string]$Prefix,
        [string]$IndexVarName
    )

    if (-not $Map.Contains($OriginalValue)) {
        $index = Get-Variable -Name $IndexVarName -Scope Script -ValueOnly
        $Map[$OriginalValue] = "{0}{1:D3}" -f $Prefix, $index
        Set-Variable -Name $IndexVarName -Scope Script -Value ($index + 1)
    }
    return $Map[$OriginalValue]
}

function New-MappingWorkbook {
    <#
        Writes a real .xlsx (Open XML) with one worksheet per mapping table.
        Built directly as a zip package, so it needs neither Excel nor the
        ImportExcel module - only .NET, which PowerShell always has.

        $Sheets is an array of hashtables: @{ Name; Headers; Rows } where Rows
        is an array of two-element arrays.
    #>
    param(
        [string]$Path,
        [object[]]$Sheets
    )

    Add-Type -AssemblyName System.IO.Compression -ErrorAction SilentlyContinue
    Add-Type -AssemblyName System.IO.Compression.FileSystem -ErrorAction SilentlyContinue

    function Format-XmlText {
        param([string]$Text)
        return [System.Security.SecurityElement]::Escape([string]$Text)
    }

    function New-SheetXml {
        param([object]$Sheet)

```

```

$builder = New-Object System.Text.StringBuilder
[void]$builder.Append('<?xml version="1.0" encoding="UTF-8" standalone="yes"?>')
[void]$builder.Append('<worksheet xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main">')
# Freeze the header row so long mappings stay readable while scrolling.
[void]$builder.Append('<sheetViews><sheetView workbookViewId="0">')
[void]$builder.Append('<pane ySplit="1" topLeftCell="A2" activePane="bottomLeft" state="frozen"/>')
[void]$builder.Append('</sheetView></sheetViews>')
[void]$builder.Append('<cols><col min="1" max="1" width="42" customWidth="1"/><col min="2" max="2" width="26" customWidth="1"/></cols>')
[void]$builder.Append('<sheetData>')

$rowIndex = 1
$allRows = @(, $Sheet.Headers) + @($Sheet.Rows)
foreach ($row in $allRows) {
    $style = if ($RowIndex -eq 1) { ' s="1"' } else { '' }
    [void]$builder.Append("&<row r=\"$RowIndex\">")
    $columnIndex = 0
    foreach ($value in $row) {
        $cellRef = "$([char](65 + $columnIndex))$RowIndex"
        [void]$builder.Append("<c r=\"$cellRef\" t=\"inlineStr\"$style><is><t xml:space=\"preserve\">$(Format-XmlText $value)</t></is></c>")
        $columnIndex++
    }
    [void]$builder.Append('</row>')
    $RowIndex++
}

[void]$builder.Append('</sheetData></worksheet>')
return $builder.ToString()
}

$sheetTags = ""
$sheetRels = ""
$contentOverrides = ""
for ($i = 0; $i -lt $Sheets.Count; $i++) {
    $id = $i + 1
    $sheetTags += "<sheet name=\"$$(Format-XmlText $Sheets[$i].Name)\" sheetId=\"$id\" r:id=\"$rId$id\"/>"
    $sheetRels += "<Relationship Id=\"$rId$id\" Type=\"http://schemas.openxmlformats.org/officeDocument/2006/relationships/worksheet\" Target=\"worksheets/sheet$id.xml\"/>"
    $contentOverrides += "<Override PartName=\"/xl/worksheets/sheet$id.xml\" ContentType=\"application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml\"/>"
}
$stylesId = $Sheets.Count + 1

$parts = [ordered]@{
    "[Content_Types].xml" = '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' +
        '<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">' +
        '<Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/>' +
        '<Default Extension="xml" ContentType="application/xml"/>' +
        '<Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/>' +
        $contentOverrides +
        '<Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/>' +
        '</Types>'

    "_rels/.rels" = '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' +
        '<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">' +
        '<Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/officeDocument" Target="xl/workbook.xml"/>' +
        '</Relationships>'

    "xl/workbook.xml" = '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' +
        '<workbook xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main" ' +
        'xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationships">' +
        "<sheets>$sheetTags</sheets></workbook>"

    "xl/_rels/workbook.xml.rels" = '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' +
        '<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">' +
        $sheetRels +
        "<Relationship Id=\"$rId$stylesId\" Type=\"http://schemas.openxmlformats.org/officeDocument/2006/relationships/styles\" Target=\"\"styles.xml\"\"/>" +
        '</Relationships>'
}

```

```

# Two cell formats: 0 = plain, 1 = bold (used for the header row).
"xl/styles.xml" = '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' +
    '<styleSheet xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main">' +
    '<fonts count="2"><font><sz val="11"/><name val="Calibri"/></font>' +
    '<font><b/><sz val="11"/><name val="Calibri"/></font></font>' +
    '<fills count="2"><fill><patternFill patternType="none"/></fill>' +
    '<fill><patternFill patternType="gray125"/></fill></fills>' +
    '<borders count="1"><border><left/><right/><top/><bottom/><diagonal/></border></borders>' +
    '<cellStyleXfs count="1"><xf numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs>' +
    '<cellXfs count="2"><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/>' +
    '<xf numFmtId="0" fontId="1" fillId="0" borderId="0" xfId="0" applyFont="1"/></cellXfs>' +
    '<cellStyles count="1"><cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles>' +
    '</styleSheet>'
}
for ($i = 0; $i -lt $Sheets.Count; $i++) {
    $parts["xl/worksheets/sheet$(($i + 1).xml)"] = New-SheetXml -Sheet $Sheets[$i]
}

# .NET resolves a relative path against the process working directory, which
# is not PowerShell's current location - always hand it a rooted path.
if (-not [System.IO.Path]::IsPathRooted($Path)) {
    $Path = Join-Path (Get-Location).ProviderPath $Path
}

if (Test-Path -LiteralPath $Path) {
    Remove-Item -LiteralPath $Path -Force
}
$encoding = New-Object System.Text.UTF8Encoding($false)
$archive = [System.IO.Compression.ZipFile]::Open($Path, [System.IO.Compression.ZipArchiveMode]::Create)
try {
    foreach ($entryName in $parts.Keys) {
        $entry = $archive.CreateEntry($entryName)
        $stream = $entry.Open()
        try {
            $writer = New-Object System.IO.StreamWriter($stream, $encoding)
            $writer.Write($parts[$entryName])
            $writer.Flush()
            $writer.Dispose()
        }
        finally {
            $stream.Dispose()
        }
    }
}
finally {
    $archive.Dispose()
}
}

$changedCount = 0
$scannedCount = 0
$totalMatchCount = 0
$totalChangedLines = 0

function Write-ChangedLines {
    param([object[]]$Lines)

    foreach ($changedLine in $Lines) {
        Write-Host ("    line {0} ({1} match(es)):" -f $changedLine.Line, $changedLine.Matches) -ForegroundColor DarkGray
        Write-Host ("        - {0}" -f $changedLine.Before) -ForegroundColor DarkRed
        Write-Host ("        + {0}" -f $changedLine.After) -ForegroundColor DarkGreen
    }
}

foreach ($fileItem in $allFiles) {
    $scannedCount++

```

```

$content = Get-Content -LiteralPath $fileItem.FullName -ErrorAction SilentlyContinue -ErrorVariable +accessErrors
if ($null -eq $content) {
    continue
}

$fileMatchCount = 0
$fileChangedLines = [System.Collections.Generic.List[object]]::new()
$lineNumber = 0

$updatedContent = foreach ($row in $content) {
    $lineNumber++
    $currentRow = $row
    $script:rowMatchCount = 0

    foreach ($search in $Replacements.Keys) {
        $lineMatches = [regex]::Matches($currentRow, $search).Count
        if ($lineMatches -gt 0) {
            $script:rowMatchCount += $lineMatches
            $currentRow = $currentRow -replace $search, $Replacements[$search]
        }
    }

    if ($StripPortDescriptions -and $portDescriptionRegex.IsMatch($currentRow)) {
        $script:rowMatchCount++
        $currentRow = $portDescriptionRegex.Replace($currentRow, '${port}')
    }

    if ($AnonymizeSwitchNames) {
        $currentRow = $switchNameRegex.Replace($currentRow, {
            param($match)
            $script:rowMatchCount++
            Get-PlaceholderName -OriginalValue $match.Value -Map $switchNameMap -Prefix $SwitchNamePrefix -IndexVarName "switchNameIndex"
        })
    }

    if ($AnonymizeUsernames) {
        $currentRow = $usernameRegex.Replace($currentRow, {
            param($match)
            $script:rowMatchCount++
            Get-PlaceholderName -OriginalValue $match.Value -Map $usernameMap -Prefix $UsernamePrefix -IndexVarName "usernameIndex"
        })
    }

    if ($AnonymizeIPs) {
        $currentRow = $ipRegex.Replace($currentRow, {
            param($match)
            $script:rowMatchCount++
            Get-PlaceholderName -OriginalValue $match.Value -Map $ipMap -Prefix $IPPrefix -IndexVarName "ipIndex"
        })
    }

    # Keep the line itself, so every change can be printed with its line
    # number and how many matches it contained.
    if ($script:rowMatchCount -gt 0) {
        $fileMatchCount += $script:rowMatchCount
        $fileChangedLines.Add([PSCustomObject]@{
            Line      = $lineNumber
            Matches    = $script:rowMatchCount
            Before     = $row.Trim()
            After      = $currentRow.Trim()
        })
    }

    $currentRow
}

```

```

    if ($fileMatchCount -eq 0) {
        continue
    }

    $changedCount++
    $totalMatchCount += $fileMatchCount
    $totalChangedLines += $fileChangedLines.Count

    if ($DryRun) {
        Write-Host "[DryRun] Would update: $($fileItem.FullName) ($fileMatchCount match(es) on $($fileChangedLines.Count) line(s))" -ForegroundColor DarkYellow
        Write-ChangedLines -Lines $fileChangedLines
        continue
    }

    if (-not $PSCmdlet.ShouldProcess($fileItem.FullName, "Replace $fileMatchCount match(es)")) {
        continue
    }

    if ($Backup) {
        $relativePath = $fileItem.FullName.Substring($resolvedRoot.Length).TrimStart('\', '/')
        $backupPath = Join-Path $BackupFolder $relativePath
        $backupDir = Split-Path -Path $backupPath -Parent
        if (-not (Test-Path -LiteralPath $backupDir)) {
            New-Item -ItemType Directory -Path $backupDir -Force | Out-Null
        }
        Copy-Item -LiteralPath $fileItem.FullName -Destination $backupPath -Force
    }

    Write-Host "Working on: $($fileItem.FullName) ($fileMatchCount match(es) on $($fileChangedLines.Count) line(s))" -ForegroundColor DarkYellow
    Write-ChangedLines -Lines $fileChangedLines
    $updatedContent | Set-Content -LiteralPath $fileItem.FullName
}

Write-Host ""
if ($DryRun) {
    Write-Host "Dry run complete. $changedCount of $scannedCount file(s) would be updated." -ForegroundColor Green
} else {
    Write-Host "Complete successfully! $changedCount of $scannedCount file(s) updated." -ForegroundColor Green
    if ($Backup) {
        Write-Host "Backups saved under: $BackupFolder" -ForegroundColor Green
    }
}

Write-Host "Total matches found: $totalMatchCount on $totalChangedLines line(s)." -ForegroundColor Green

function Write-NameMapping {
    param([string]$Title, [System.Collections.Specialized.OrderedDictionary]$Map)

    if ($Map.Count -eq 0) {
        return
    }
    Write-Host "`n$Title mapping for this run:" -ForegroundColor Cyan
    $Map.GetEnumerator() | ForEach-Object {
        Write-Host (" {0} -> {1}" -f $_.Key, $_.Value)
    }
}

if ($AnonymizeSwitchNames) { Write-NameMapping -Title "Switch name" -Map $switchNameMap }
if ($AnonymizeUsernames) { Write-NameMapping -Title "Username" -Map $usernameMap }
if ($AnonymizeIPs) { Write-NameMapping -Title "IP address" -Map $ipMap }

# Excel workbook with one tab per mapping table, so the "who became what" list
# survives the run instead of only scrolling past in the console.
if ($ExportMappings -or $ExcelPath) {
    if (-not $ExcelPath) {

```

```

    $ExcelPath = Join-Path (Resolve-Path -LiteralPath $Path) "Redaction-Mappings_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').xlsx"
}

$mappingSheets = @(
    @(
        @{ Name = "Switches";      Headers = @("Original Switch Name", "New Name"); Map = $switchNameMap }
        @{ Name = "Users";        Headers = @("Original User Name", "New Name");   Map = $usernameMap }
        @{ Name = "IP Addresses"; Headers = @("Original IP Address", "New Name");   Map = $ipMap }
    ) | ForEach-Object {
        @{
            Name      = $_.Name
            Headers    = $_.Headers
            Rows       = @($_.Map.GetEnumerator() | ForEach-Object { , @($_.Key, $_.Value) })
        }
    }
)

try {
    New-MappingWorkbook -Path $ExcelPath -Sheets $mappingSheets
    $rowTotal = ($mappingSheets | ForEach-Object { @($_.Rows).Count } | Measure-Object -Sum).Sum
    Write-Host "`nMapping workbook saved to: $ExcelPath ($rowTotal mapping(s) across 3 tabs)" -ForegroundColor Cyan
}
catch {
    Write-Warning "Could not write the mapping workbook to '$ExcelPath': $($_.Exception.Message)"
}

}

# Folders and files that could not be opened, with the reason. TargetObject holds
# the path for these provider errors; fall back to the error's target name.
$inaccessible = $accessErrors |
    ForEach-Object {
        [PSCustomObject]@{
            Path      = if ($_.TargetObject) { $_.TargetObject } else { $_.CategoryInfo.TargetName }
            Reason     = $_.Exception.Message
        }
    } |
    Where-Object { $_.Path } |
    Sort-Object Path -Unique

if ($inaccessible) {
    Write-Host "`nPaths that could not be accessed and were skipped: $($($inaccessible).Count)" -ForegroundColor Yellow
    $inaccessible | Format-Table -AutoSize -Wrap
}

```